



ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia
a informatiky

Semestrálna práca z predmetu
vývoj aplikácií pre mobilné zariadenia

VISUAL TASK ORGANIZER

Vypracoval: Tomáš Sobek

Študijná skupina: 5ZYI23

Akademický rok: 2025/26

V Žiline dňa 12.4.2026



Obsah

Cieľ aplikácie.....	2
Diagram prípadov použitia	2
Diagramy návrhu riešenia	3
Popis implementácie	5
Výsledná aplikácia – snímky z emulátora	8
Zoznam zdrojov	11

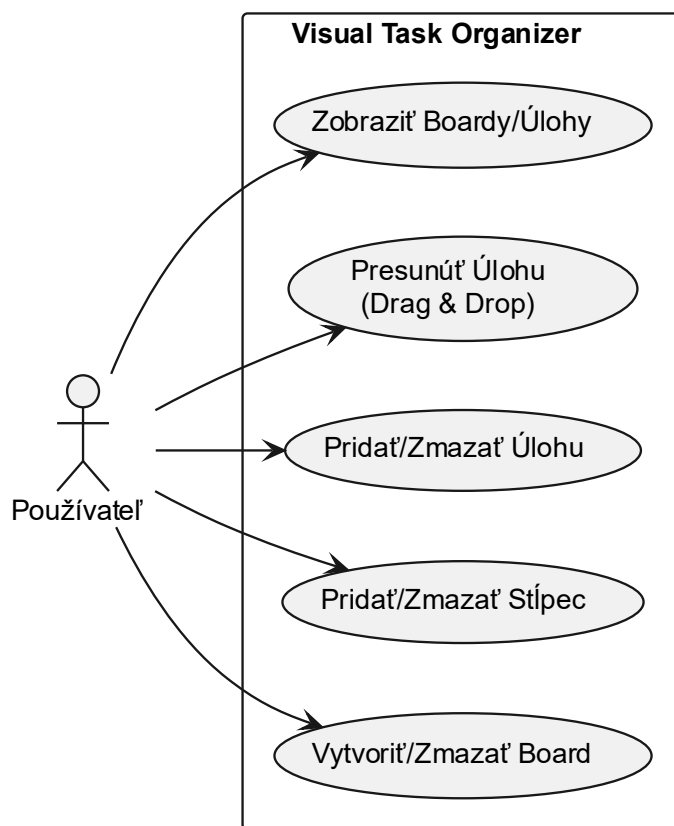
Cieľ aplikácie

Cieľom semestrálneho projektu bolo navrhnuť a implementovať mobilnú aplikáciu pre operačný systém Android, ktorá slúži na organizáciu úloh a projektov pomocou vizuálnej metodiky Kanban. Aplikácia bola navrhnutá tak, aby poskytla používateľovi intuitívny nástroj na evidenciu práce, sledovanie progresu a správu termínov.

Hlavné funkčné aspekty aplikácie:

- **Vizuálne manažovanie úloh:** Používateľ má možnosť vytvárať si vlastné boardy, v ktorých si definuje štruktúru stĺpcov podľa svojich potrieb (napr. "To Do", "In Progress", "Done").
- **Drag-and-Drop manipulácia:** Kľúčovou vlastnosťou aplikácie je podpora presúvania úloh medzi jednotlivými stĺpcami pomocou dotykového ovládania, čo umožňuje rýchlu zmenu stavu úlohy a okamžitú vizuálnu spätnú väzbu.
- **Kategorizácia a prioritizácia:** Každá úloha môže obsahovať popis, dátum začiatku, termín dokončenia (deadline), označenie priority a vlastné štítky (labels), čo používateľovi pomáha pri lepšej orientácii v úlohách.
- **Offline perzistencia:** Všetky dáta sú ukladané lokálne v zariadení, čo zabezpečuje prístup k úlohám aj bez internetového pripojenia.

Diagram prípadov použitia



Obrázok 1 - Diagram prípadov použitia definuje hlavné interakcie používateľa so systémom

Diagramy návrhu riešenia

Entitno-relačná schéma (ERD) znázorňuje štruktúru relačnej databázy, ktorá zabezpečuje perzistenciu dát v aplikácii.

Návrh využíva hierarchickú väzbu 'jedna k mnohým' (1:N) medzi entitami Boards, Columns a Tasks.

Board (Nástenka): Predstavuje najvyššiu úroveň hierarchie a slúži ako kontajner pre celý projekt alebo tematickú oblasť. Obsahuje základné metadáta projektu, konkrétne názov a identifikátor farebnej témy, ktorá vizuálne odlišuje jednotlivé boardy v prehľade.

Column (Stĺpec): Definuje jednotlivé fázy procesu (napr. „To Do“, „In Progress“, „Done“). Každý stĺpec je priradený ku konkrétnemu boardu (boardId) a disponuje poradovým číslom (orderIdIndex), ktoré určuje jeho vizuálne usporiadanie na obrazovke. Slúži ako priamy kontajner pre entity typu Task.

Task (Úloha): Predstavuje najnižšiu úroveň hierarchie a reprezentuje konkrétnu pracovnú jednotku. Obsahuje komplexné informácie o úlohe (názov, popis, časové údaje začiatku a termínu, prioritu, farebný štítok a stav dokončenia). Vďaka väzbe columnId je úloha priradená k danému stĺpcu, čo umožňuje jej následnú manipuláciu a zmenu stavu v rámci workflow.

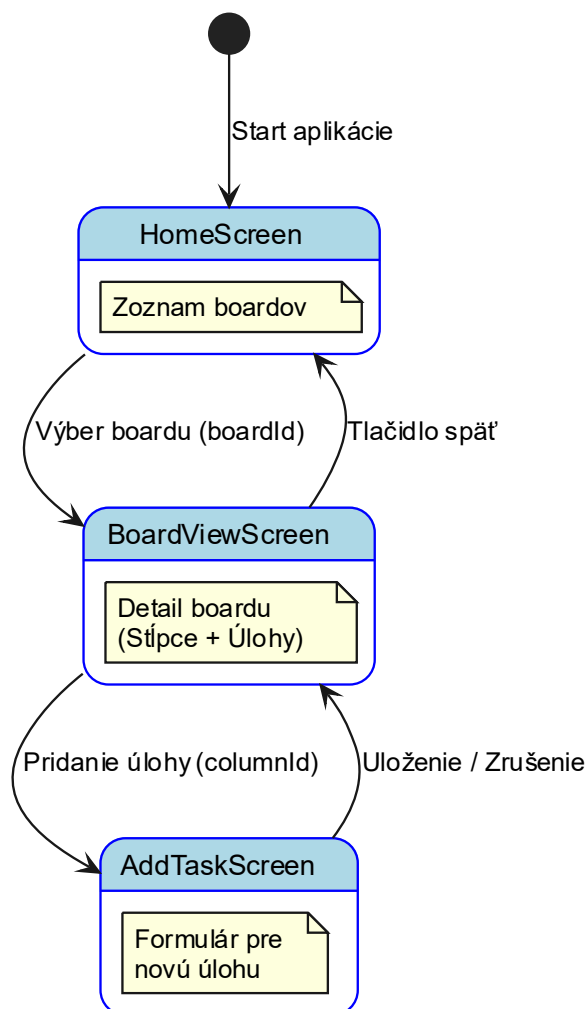
Implementácia cudzích kľúčov (Foreign Keys) s nastavením 'CASCADE' zaisťuje referenčnú integritu – pri vymazaní nadradeného záznamu (board alebo stĺpec) sa automaticky odstránia aj všetky podriadené záznamy, čo predchádza vzniku nekonzistentných dát v databáze



Obrázok 2- entitno-relačný diagram

Diagram tokov obrazoviek mapuje hierarchiu navigácie v aplikácii a definuje používateľskú cestu (User Journey).

Diagram ilustruje prechody medzi hlavnými komponentmi (HomeScreen, BoardViewScreen, AddTaskScreen) a zdôrazňuje využitie Android Navigation Componentu.



Obrázok 3- Diagram toku obrazoviek (screen flow)



Popis implementácie

Aplikácia bola vyvinutá pre platformu Android s využitím compileSdk verzie 36. Počas vývoja a testovania bol využitý emulátor „Medium Phone“ s operačným systémom Android 15 (API 35), čím je zabezpečená plná kompatibilita s modernými štandardmi platformy.

Štruktúra projektu je navrhnutá modulárne, s rozdelením do logických balíkov (**packages**), ktoré zabezpečujú prehľadnosť a ľahkú údržbu kódu:

- **data.local**: Obsahuje definíciu databázy (AppDatabase) a rozhrania pre prístup k dátam (BoardDao, ColumnDao, TaskDao).
- **data.repository**: Spravuje logiku prístupu k dátam, čím oddeľuje databázovú vrstvu od aplikačnej logiky.
- **model**: Zahŕňa dátové entity (Board, Column, Task), ktoré tvoria jadro systému.
- **ui** je navrhnutý podľa princípov modulárneho vývoja v Jetpack Compose.
- **ui.board**: Obsahuje hlavnú obrazovku BoardViewScreen pre zobrazenie obsahu nástenky, formulár AddTaskScreen pre vytváranie nových úloh a BoardViewModel, ktorý riadi biznis logiku a stav dát pre danú časť aplikácie.
- **ui.home**: Obsahuje HomeScreen, ktorá slúži ako vstupný bod aplikácie a poskytuje používateľovi prehľad a správu všetkých vytvorených násteniek (boardov).
- **ui.navigation**: Obsahuje AppNavigation, centrálny bod, kde je definovaný navigačný graf (NavHost). Tento komponent riadi všetky prechody medzi obrazovkami a zabezpečuje správnu distribúciu parametrov pri navigácii.
- **ui.task**: Obsahuje triedu TaskCard, ktorá slúži na vizuálne zobrazenie informácií o konkrétnej úlohe v rámci zoznamov.
- **ui.theme**: Zastrešuje vizuálnu identitu aplikácie prostredníctvom súborov Color.kt, Theme.kt a Type.kt. Táto vrstva zabezpečuje konzistentný vzhľad aplikácie v súlade s dizajnovými systémami (Material Design) a umožňuje jednoduchú úpravu typografie či farebnej palety. Avšak v implementácii tohto projektu nebola upravená od vytvorenia projektu cez šablónu Empty activity.

Nasledujúce časti podrobne opisujú implementáciu jednotlivých kľúčových komponentov.

3.1 Architektúra aplikácie (MVVM Pattern)

Aplikácia implementuje architektúru MVVM (Model-View-ViewModel), ktorá zabezpečuje striktné oddelenie logiky od používateľského rozhrania.

- ViewModel (BoardViewModel): Pôsobí ako sprostredkovateľ medzi UI a dátovou vrstvou. Uchováva stav aplikácie (State) a spracováva používateľské akcie. Vďaka ViewModel ostáva UI vrstva odolná voči zmenám konfigurácie (napr. rotácia zariadenia), pretože stav pretrváva v pamäti.
- Repository Pattern: Využité boli repozitáre (BoardRepository, TaskRepository, ColumnRepository), ktoré abstrahujú prístup k dátam. ViewModel nekomunikuje priamo s databázou, ale volá metódy repozitára, čo umožňuje jednoduché testovanie a budúcu rozšíriteľnosť (napr. pridanie sieťovej komunikácie).

3.2 Perzistencia dát (Room Database)

Na ukladanie dát sme využili knižnicu Room, ktorá je nadstavbou nad SQLite.

- Entity: Definované modely (Board, Column, Task) sú označené anotáciou @Entity. Pre zabezpečenie integrity dát boli použité ForeignKey väzby s kaskádovým mazaním (CASCADE), čo zaručuje, že pri odstránení boardu sa automaticky vymažú aj všetky jeho stĺpce a úlohy.
- DAO (Data Access Objects): Komunikácia s databázou prebieha cez rozhrania DAO (BoardDao, TaskDao, ColumnDao), ktoré využívajú Kotlin Flows. Vďaka tomu je UI automaticky notifikované pri každej zmene v databáze (reaktívnosť).

3.3 Používateľské rozhranie (Jetpack Compose)

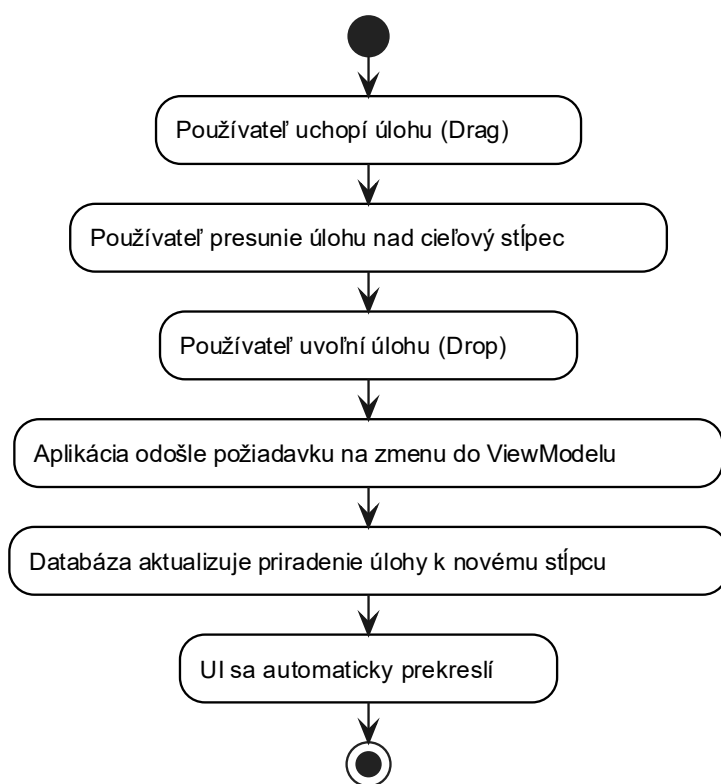
UI je postavené na knižnici Jetpack Compose, ktorá umožňuje deklaratívne vytváranie komponentov.

- Reaktívny stav: Využité funkcie ako collectAsState(), ktoré automaticky prekresľujú UI komponenty (napr. TaskCard alebo ColumnComponent) vždy, keď sa zmenia dáta v databáze.
- Optimalizácia zoznamov: Pri zobrazení veľkého množstva dát boli použité LazyRow a LazyColumn, ktoré zabezpečujú efektívne vykresľovanie len tých prvkov, ktoré sú práve viditeľné na obrazovke, čím sa minimalizuje spotreba pamäte.

3.4 Implementácia Drag & Drop

Funkcionalita Drag & Drop umožňuje používateľom intuitívne meniť stav úloh ich presúvaním medzi jednotlivými stĺpcami. Celý proces je implementovaný v troch nadväzujúcich krokoch:

1. Iniciácia: Používateľ dlhým stlačením konkrétnej úlohy (Drag) aktivuje režim presunu. Aplikácia v tomto momente identifikuje unikátne ID danej úlohy.
2. Presun a cieľ: Používateľ presunie úlohu nad cieľový stĺpec. Aplikácia priebežne monitoruje polohu kurzora a identifikuje cieľovú oblasť.
3. Aktualizácia dát: Po uvoľnení úlohy (Drop) v novom stĺpci sa informácia o presune odošle do ViewModelu. Ten následne aktualizuje stĺpec danej úlohy v databáze. Vďaka reaktívnemu prepojeniu UI s databázou (Flow) sa zoznam úloh po zápise do databázy automaticky prekreslí a úloha sa okamžite zobrazí v novom stĺpci.



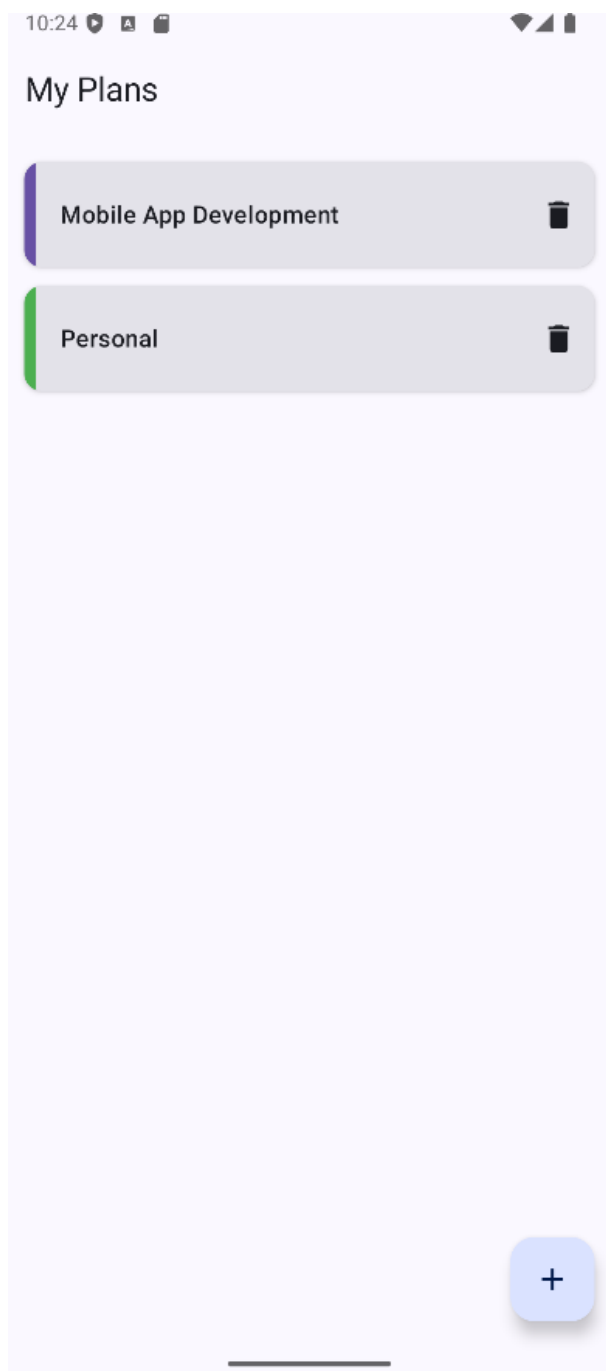
Obrázok 4- diagram funkcionality drag & drop

3.5 Navigácia

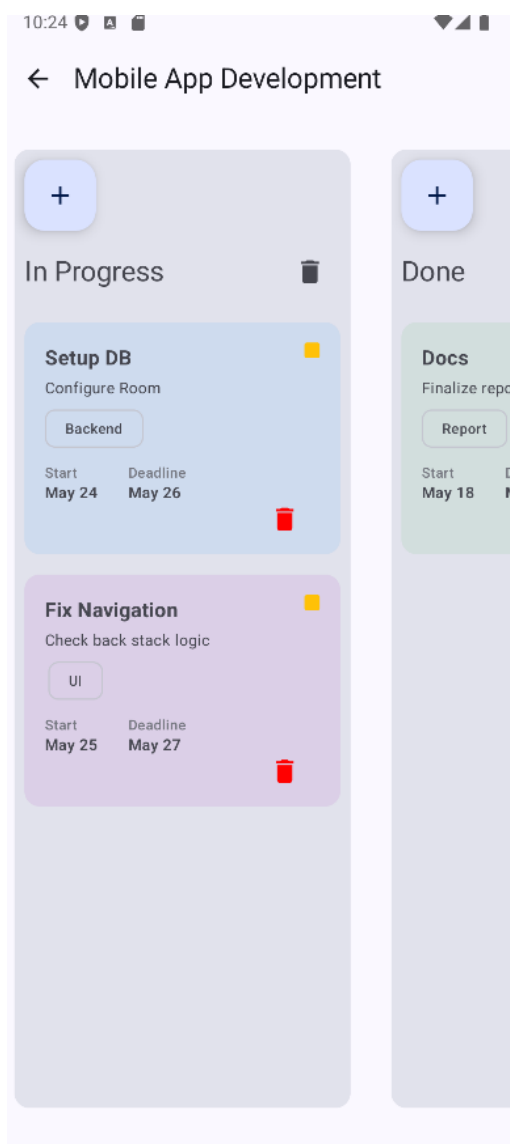
Navigácia medzi obrazovkami je riešená pomocou Navigation Component (NavHost, composable). Pri prechodoch medzi obrazovkami (napr. z HomeScreen na BoardViewScreen) sa odovzdávajú identifikátory cez argumenty (navArgument), čo zabezpečuje, že každá obrazovka je nezávislá a pracuje s presne definovaným kontextom (ID boardu alebo stĺpca).



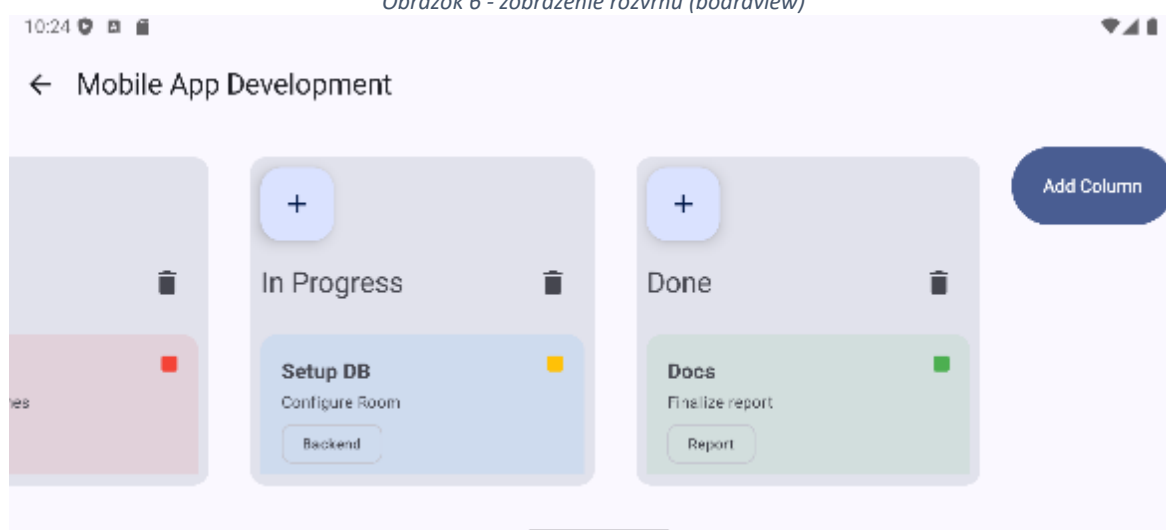
Výsledná aplikácia – snímky z emulátora



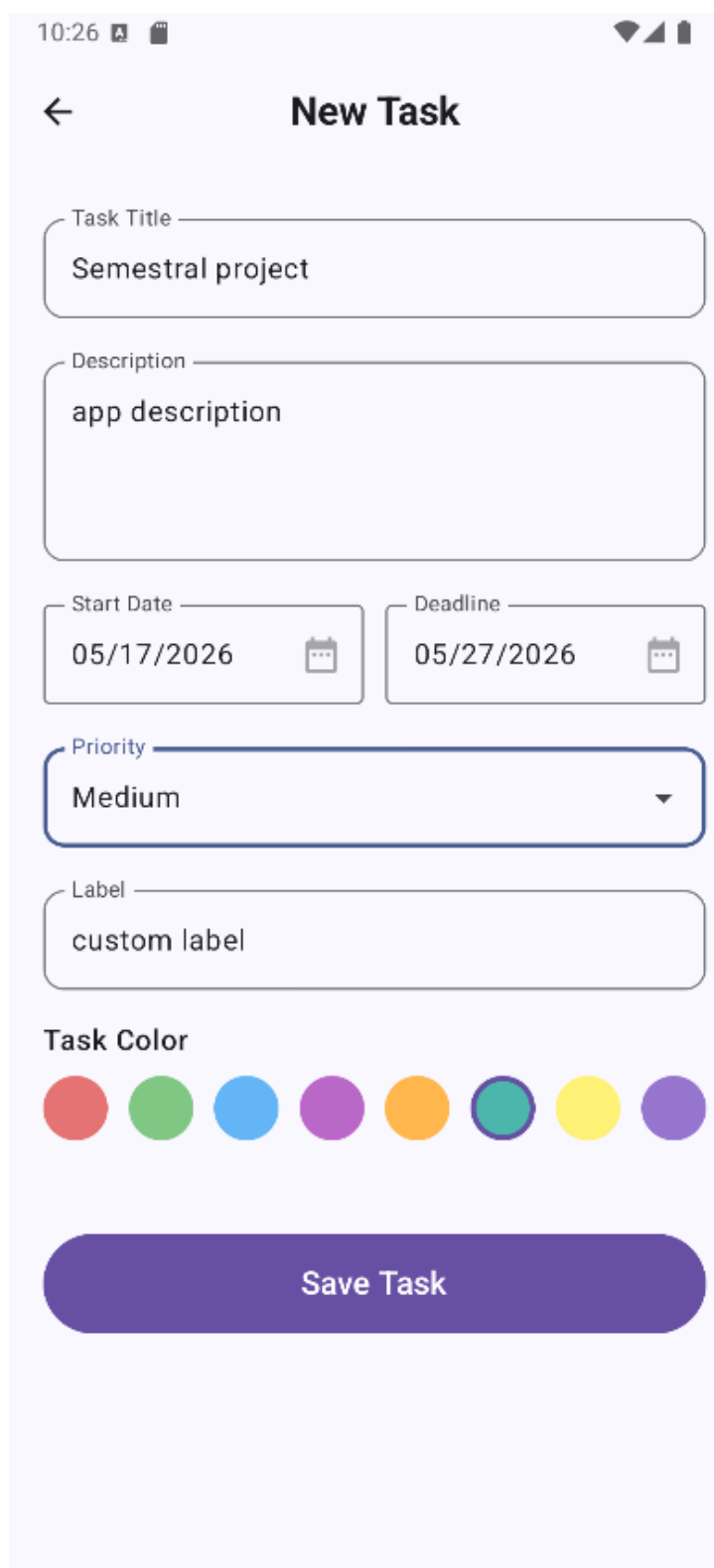
Obrázok 5 - úvodná obrazovka (homescreen)



Obrázok 6 - zobrazenie rozvrhu (boardview)



Obrázok 7 - zobrazenie rozvrhu v horizontálnom režime



The screenshot shows a mobile application interface for creating a new task. At the top, the status bar displays the time 10:26 and battery level. The app's title bar shows a back arrow and the text 'New Task'. The form consists of several input fields: 'Task Title' with the value 'Semestral project', 'Description' with the value 'app description', 'Start Date' with the value '05/17/2026' and a calendar icon, 'Deadline' with the value '05/27/2026' and a calendar icon, 'Priority' with a dropdown menu showing 'Medium', and 'Label' with the value 'custom label'. Below these fields is a 'Task Color' section with eight colored circles: red, green, blue, purple, orange, teal, yellow, and dark purple. At the bottom is a large purple button labeled 'Save Task'.

Obrázok 8- obrazovka pridania novej úlohy (task)



Zoznam zdrojov

- ❓ **ANDROID DEVELOPERS.** *Guide to app architecture.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://developer.android.com/topic/architecture>
- ❓ **ANDROID DEVELOPERS.** *Jetpack Compose UI.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://developer.android.com/compose>
- ❓ **ANDROID DEVELOPERS.** *Save data in a local database using Room.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://developer.android.com/training/data-storage/room>
- ❓ **DBDIAGRAM.IO.** *Database Relationship Design Tool.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://dbdiagram.io>
- ❓ **KOTLIN.** *Coroutines Guide.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://kotlinlang.org/docs/coroutines-overview.html>
- ❓ **MATERIAL DESIGN.** *Material Design 3 Guidelines.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://m3.material.io>
- ❓ **PLANTUML.** *PlantUML Language Reference Guide.* [online]. [Citované 25. 5. 2026]. Dostupné na: <https://plantuml.com>
- ŠTUDIJNÉ MATERIÁLY.** *[Vývoj aplikácií pre mobilné zariadenia]: Prezentácie z prednášok a materiály k cvičeniam.* [online]. [Citované 25. 5. 2026].